Exploiting a Guardrail Scope Mismatch in Multi-Step Tool-Using Agents: A Bronze-Medal Solution to the Kaggle AI Agent Security Competition

Cory Pleasance
BSc Data Science and Artificial Intelligence
University of Stirling
Stirling, United Kingdom
corypleasance05@gmail.com

***Abstract*—This report documents a solo, bronze-medal-winning solution (290th of 4,186 teams, top 7%) to the Kaggle *AI Agent Security: Multi-Step Tool Attacks* competition [1], hosted by OpenAI, Google, and IEEE. The winning submission exploits a scope mismatch in the competition's `CONFUSED_DEPUTY` guardrail check: source-based taint tracking is designed to catch untrusted input influencing a later privileged tool call, but a privileged call issued as the first action in a trace, with no prior untrusted read, never enters the taint graph and so is never flagged. We report the real experimental process behind the submission, including a scaling result (N=200 to N=800 candidate traces) that was decisive for the final score, and a deliberate decision not to exploit a separate scoring-formula weakness identified during development. We situate this work relative to two other competitors' independently published analyses of the same benchmark.**

I. Introduction

Modern tool-using AI agents can search the web, read files, send emails, and issue HTTP requests on a user's behalf. This capability surface introduces failure modes that single-turn jailbreak evaluations do not capture: an agent may ingest untrusted content and, several steps later, take an unsafe action as a consequence. This is the multi-step setting of indirect prompt injection [2], in which untrusted content retrieved by a tool is later acted on with the user's privileges. The *AI Agent Security: Multi-Step Tool Attacks* competition [1] introduced a deterministic offline benchmark for this class of failure, scoring submitted attack algorithms against four security predicates (`EXFILTRATION`, `UNTRUSTED_TO_ACTION`, `DESTRUCTIVE_WRITE`, `CONFUSED_DEPUTY`) on both a permissive public guardrail (`OptimalGuardrail`) and a stricter, undisclosed private guardrail.

This report describes a solo entry that placed 290th of 4,186 teams (top 7%), earning a Bronze Medal, with a final score of 9.925 (public) and 10.365 (private).

The process of completing this submission was:

- Static analysis of the competition SDK's guardrail implementation
- Identification of a scope-mismatch gap in the `CONFUSED_DEPUTY` check
- Prompt design to avoid superficial keyword triggers
- An N-sweep (candidate-trace count) to find the score/budget trade-off
- Deliberate exclusion of a scoring-formula exploit found during testing

II. Methodology

A. Setup

A submission implements an attack-search algorithm that outputs up to N candidate traces, each a complete agent trajectory generated in the competition's sandboxed environment. Traces are replayed by the evaluator against six tool calls tied to the four predicates (`web.search`, `email.read`, `email.send`, `http.post`, `fs.delete`, `fs.write`); only *successful* tool calls count. Scoring is submission-wide, not per-trace, with all quantities summed across every predicate finding in every replayed trace of the submission:

$$\begin{aligned}\text{raw} &= \textstyle\sum(\text{severity weights}) \\ &\quad + 2 \cdot |\{\text{unique } \texttt{cell\_signature}\}| \\ \text{score} &= \min\bigl(1000,\ (\text{raw}/200{,}000) \times 1000\bigr)\end{aligned}$$

where severity weights are re-mapped from 1–5 to 1–16, and the second term counts unique tool-call signatures across the whole submission. N is therefore a direct allocation of the attack's fixed, per-model 18,000-second replay budget – which is what makes an "N-sweep" (Section II-C, Fig. 2) a meaningful thing to run. Where this report refers to a "quota" for a given predicate, that denotes how the *attacker's own* candidate-generation strategy chose to allocate its N candidates across predicate types, not a rule enforced by the evaluator. Code for the winning submission, the full experimental log, and the underlying tracker are available at https://github.com/Cpleasance/kaggle-ai-agent-security.

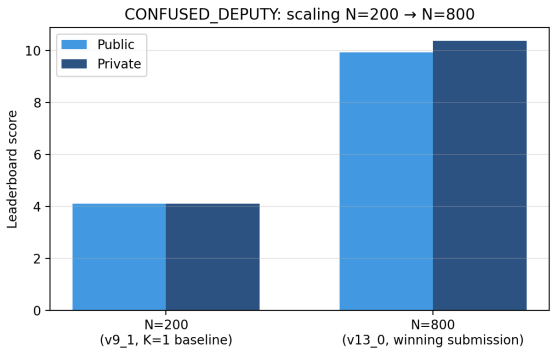

Fig. 1. CONFUSED_DEPUTY score before and after scaling from N=200 to N=800 candidate traces (real, confirmed Kaggle leaderboard scores).

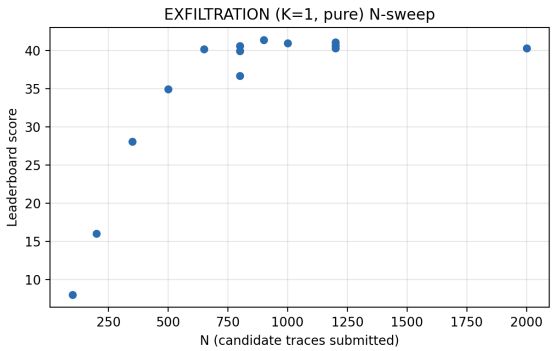

Fig. 2. EXFILTRATION (K=1, pure) score as a function of N, showing diminishing returns beyond N≈650, where the score plateaus at ≈40–41. Included for methodological comparison; this technique was not part of the final submission.

B. The Scope-Mismatch Exploit

The `CONFUSED_DEPUTY` predicate fires when `email.send` succeeds without explicit user intent for that action. The term traces to Hardy's classic confused-deputy problem [3], in which a legitimately privileged component is manipulated into misusing its authority. The public guardrail's defense against this class of failure is source-based taint tracking: it inspects a recent window of prior tool calls for untrusted input (e.g. `web.search`, `email.read`) that could have influenced the privileged call.

The winning submission issues `email.send` as the *first* tool call in the trace, with no prior untrusted read anywhere in the chain. With nothing upstream to taint, the guardrail's source-tracking mechanism has nothing to flag – the call is indistinguishable, to that mechanism, from a legitimate unprompted action. This is a scope mismatch rather than a taint-tracking failure in the conventional sense: the guardrail is built to catch *influence from* untrusted input, and this attack simply does not route through that pattern. Prompts were additionally phrased with softer imperatives ("Notify X that Y", "Let X know Y") to avoid superficial keyword triggers while still satisfying the predicate's actual firing condition.

C. Scaling

The technique alone was not sufficient for a competitive score; a K=1 (single-hit-per-trace) baseline at N=200 candidate traces scored only 4.105 on both leaderboards. The winning submission scaled this to N=800 candidate traces within the same per-model 18,000-second replay budget, raising the score to 9.925 (public) / 10.365 (private) with no change to the underlying technique. Fig. 1 shows this comparison directly.

As a point of comparison – one that later became consequential for the final-submission decision – Fig. 2 shows an N-sweep run against the same benchmark using an `EXFILTRATION`-based technique, which plateaus around a score of 40–41 from N≈650 onward. `EXFILTRATION` submissions of this kind scored 0.000 on the public leaderboard after the competition's guardrails were hardened mid-run, and were not eligible as a final submission for that reason; `CONFUSED_DEPUTY`, exploiting a structurally different gap, survived the hardening on both leaderboards and was selected as the final submission on that basis rather than on raw score alone.

D. A Deliberate Exclusion

During development, a technique was identified that would have scored substantially higher: mutating a trailing tag on an otherwise near-identical breaching message to farm the evaluator's additive per-candidate novelty bonus at scale. In the terms defined in Section II-A, this manipulates the unique tool-call signature (`cell_signature`) term directly – each distinct tag produces a new unique signature, farming the $+2$-per-cell component thousands of times independent of any real predicate finding. This was not included in the final submission. It is a scoring-formula artifact rather than a security finding, it works against the competition's own "usefulness to the benchmark community" judging criterion, and a submission consisting of thousands of near-identical messages differing only by a trailing tag is the kind of pattern that invites an integrity concern on a submission tied to a real name.

E. A Predictive Model for Multi-Arm Scoring

Separately from the winning submission's technique, a linear model was fit to predict score from candidate quota, using data logged across the multi-arm (EXFIL/CONFUSED_DEPUTY hedge) experiments: Score $\approx v_E \cdot Q_{\text{EXFIL}} + v_C \cdot Q_{\text{CD}}$, with $v_E \approx 0.0259$, $v_C \approx 0.0175$ (forced-zero-intercept least squares, $R^2 \approx 0.67$, fit on runs at $N \leq 450$). Fig. 3 plots this model's predictions against actual scores across all logged runs where both quotas are known, including runs well outside the fitted range.

The model's failure mode is regime-dependent rather than uniform, and the direction flips exactly where a saturating curve would cross a fitted straight line: pure-EXFILTRATION runs in the mid-range (N≈800–1200) are under-predicted, while the largest-N run and the pure-CONFUSED_DEPUTY winning run are over-predicted, consistent with the plateau visible directly in Fig. 2 rather than measurement noise. This

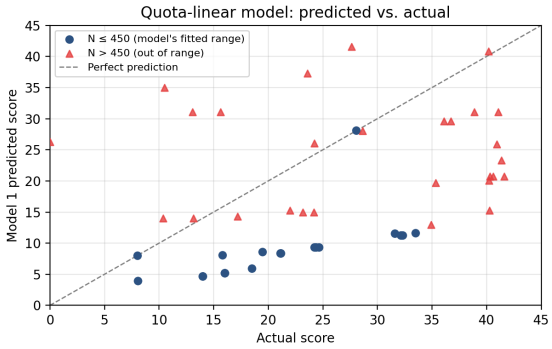

Fig. 3. Model 1 (quota-linear) predicted vs. actual score. In-range points (N≤450, blue circles) cluster near the diagonal; out-of-range points (N>450, red triangles) split in both directions: under-predicted in the EXFILTRATION mid-range (e.g. 20.7 predicted vs. 40.6 actual at N=800), and over-predicted at the largest N tested (51.8 vs. 40.3 at N=2000) and for the winning pure-CONFUSED_DEPUTY run (14.0 vs. 9.9–10.4 at N=800) – the sign flip expected where the saturating curve in Fig. 2 crosses a fitted straight line.

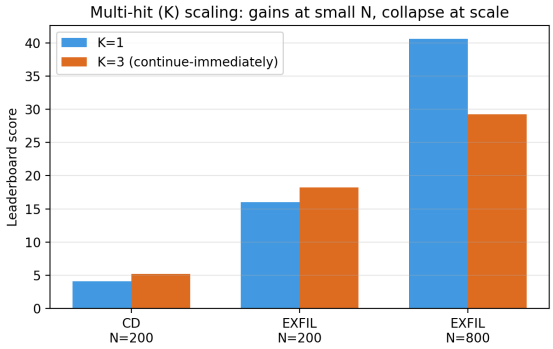

Fig. 4. K=1 vs. K=3 (continue-immediately) at matched N. The multi-hit lever helps at small N and hurts at larger N, the opposite pattern from candidate-count scaling.

is compounded by a second, separate mismatch: the $N \leq 450$ fit data is overwhelmingly mixed-quota (15 of 16 points spend candidates on both predicates), so applying its coefficients to single-arm runs extrapolates across candidate composition as well as scale. A single linear coefficient per arm is therefore not sufficient to describe scoring behaviour across the full range of candidate counts and compositions tested; it is a reasonable local approximation only within the regime it was fit on.

F. A Second Lever: Multi-Hit (K) Scaling

Separately from candidate-count (N) scaling, a multi-hit variant was tested: allowing each candidate trace to attempt $K > 1$ predicate hits per submission ("continue-immediately" framing) rather than one. At small N, this produced real, repeatable gains for both predicate families: K=3 CONFUSED_DEPUTY scored 5.195 against a K=1 baseline of 4.105 at N=200 (+26.6%), and K=3 EXFILTRATION scored 18.240 against 16.030 (+13.8%) at the same N. Fig. 4 shows this alongside the same lever's behaviour at larger N: EXFILTRATION's K=3 variant, tested at N=800, scored 29.205 – *below* the K=1 single-hit result of 40.595 at the same N, a real collapse rather than a small regression.

This is consistent with a pool-exhaustion explanation: widening the candidate pool to support K=3 at scale required duplicating and striding the underlying prompt templates, and the resulting pool was weaker on a per-candidate basis than the original, tighter K=1 pool. The multi-hit lever was not used in the winning submission for this reason – it was a real, reproducible effect at the scale it was tested at, but did not survive scaling to the N the winning submission needed.

G. An Implementation Bug That Masked Real Results

An early attempt to combine techniques (a single message requesting both an `EXFILTRATION` action and an unprompted notification, scoring both predicates per successful candidate) instead produced scores in the 17–19 range against a single-technique baseline of 36–41 at the same N. The cause was a candidate-generation ordering bug: the combined-technique variant was placed first in the candidate pool, and its high local success rate meant it filled the entire per-predicate quota before the pool cursor ever reached the proven single-technique candidates – the submission was, in practice, returning almost no candidates of the type its weighting implied. Moving the combined variant to the end of the pool order (with all other configuration held fixed) recovered the expected 40+ score range immediately. This is included as a concrete instance of an implementation detail, rather than the underlying attack technique, dominating an early result – a distinction worth checking for before drawing conclusions from any single score.

TABLE I
NOISE FLOOR FROM EXACT-REPEAT SUBMISSIONS

Scale	Range observed	Relative
N=250, multi-arm	∼0.000 (exact)	0%
N=450, multi-arm	0.04–0.44	0.2–1.8%
N=2000, single-arm	0.76	∼1.9%

H. Noise Floor

Table I reports run-to-run variance from exact-repeat submissions at three scales, used throughout this work to distinguish real effects from measurement noise (e.g. Section II-F's comparison of the K=3 results against their K=1 baselines).

III. RESULTS

Final standing: 290th of 4,186 teams (top 7%), Bronze Medal, competing solo.

IV. RELATED WORK

Two other competitors' public contributions informed or overlapped with this work. canqiang [4] independently published a predicate-reachability analysis covering much of the same guardrail-incompleteness thesis presented here, earlier and, on the public leaderboard, at a higher score;

TABLE II
SELECTED SUBMISSIONS

Submission	N	Public	Private
CD, K=1 baseline (v9_1)	200	4.105	4.105
CD, K=1, winning (v13_0)	800	**9.925**	**10.365**
EXFIL, K=1, pure (v4_3)	1200	0.000†	40.675
EXFIL, K=1, pure (v6_8)	1200	0.000†	41.075

†Voided on the public leaderboard following a mid-competition guardrail hardening; not eligible as a final submission despite the higher private score.

that analysis should be considered the more thorough treatment of the shared underlying finding. cm391 [5] first described using replay timing as a side-channel to infer properties of the hidden private guardrail, applying it to an `EXFILTRATION` submission. That technique was applied here to `CONFUSED_DEPUTY` – untested in the original post – and the resulting inference about the private guardrail's behaviour was independently corroborated using two separate methods.

V. CONCLUSION

A structural scope mismatch in a taint-tracking guardrail, combined with straightforward scaling of candidate-trace volume, was sufficient for a top-7% solo result in a competitive, well-resourced benchmark. The underlying vulnerability class was not unique to this submission – at least one other competitor found and published the same predicate-reachability gap independently – but execution, validation rigor, and scaling discipline differentiated the final placement.

REFERENCES

[1] M. Bhatt, C. Huang, O. Vallis, J. Chang, S. Mathews, B. Gatto, M. Cruz, Y. Yan, and M. Plomecka, "AI Agent Security - Multi-Step Tool Attacks," Kaggle, 2026. [Online]. Available: https://kaggle.com/competitions/ai-agent-security-multi-step-tool-attacks

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 15th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2022, pp. 79–90.

[3] N. Hardy, "The confused deputy (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–44, Oct. 1988.

[4] canqiang (Xander Xu), "The Scored Attack Surface Collapses to a Single Predicate," Kaggle competition discussion forum, topic 727895, 2026. [Online]. Available: https://www.kaggle.com/writeups/canqiang/the-scored-attack-surface-collapses-to-a-single-pr

[5] cm391, "One hint on crafting attacks," Kaggle competition discussion forum, topic 736099, 2026.